

*Федеральное государственное автономное образовательное
учреждение высшего образования «Национальный исследовательский
университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники



Лабораторная работа №1 по дисциплине «Вычислительная
математика»
Вариант 10

Выполнил:
Студент группы Р3212
Малышев Никита Александрович
Преподаватель:
Машина Екатерина Сергеевна

г. Санкт-Петербург
2025

Цель работы

Написать алгоритм для решения СЛАУ посредством алгоритма Гаусса-Зейделя на языке программирования Python.

Описание метода

Метод Гаусса-Зейделя является модификацией метода простой итерации и обеспечивает более быструю сходимость к решению систем уравнений.

Так же как и в методе простых итераций строится эквивалентная СЛАУ и за начальное приближение принимается вектор правых частей (как правило, но может быть выбран и нулевой, и единичный вектор): $x_i^0 = (d_1, d_2, \dots, d_n)$.

$$\begin{aligned}x_1 &= c_{11}x_1 + c_{12}x_2 + \dots + c_{1n}x_n + d_1 \\x_2 &= c_{21}x_1 + c_{22}x_2 + \dots + c_{2n}x_n + d_2 \\&\dots \dots \\x_n &= c_{n1}x_1 + c_{n2}x_2 + \dots + c_{nn}x_n + d_n\end{aligned}$$

Идея метода: при вычислении компонента $x_i^{(k+1)}$ на $(k+1)$ -й итерации используются $x_1^{(k+1)}, x_2^{(k+1)}, \dots, x_{i-1}^{(k+1)}$, уже вычисленные на $(k+1)$ -й итерации.

Значения остальных компонент $x_{i+1}^{(k+1)}, x_{i+2}^{(k+1)}, \dots, x_n^{(k+1)}$ берутся из предыдущей итерации.

Схема для $k = 1$:

$$\begin{aligned}x_1^1 &\rightarrow x_2^0 & x_3^0 & x_4^0 \\x_2^1 &\rightarrow x_1^1 & x_3^0 & x_4^0 \\x_3^1 &\rightarrow x_1^1 & x_2^1 & x_4^0 \\x_4^1 &\rightarrow x_1^1 & x_2^1 & x_3^1\end{aligned}$$

Схема для $k = 2$:

$$\begin{aligned}x_1^2 &\rightarrow x_2^1 & x_3^1 & x_4^1 \\x_2^2 &\rightarrow x_1^2 & x_3^1 & x_4^1 \\x_3^2 &\rightarrow x_1^2 & x_2^2 & x_4^1 \\x_4^2 &\rightarrow x_1^2 & x_2^2 & x_3^2\end{aligned}$$

$$\begin{aligned} x_1^{(k+1)} &= c_{11}x_1^{(k)} + c_{12}x_2^{(k)} + \dots + c_{1n}x_n^{(k)} + d_1 \\ x_2^{(k+1)} &= c_{21}x_1^{(k+1)} + c_{22}x_2^{(k)} + \dots + c_{2n}x_n^{(k)} + d_2 \\ x_3^{(k+1)} &= c_{31}x_1^{(k+1)} + c_{32}x_2^{(k+1)} + c_{33}x_3^{(k)} \dots + c_{3n}x_n^{(k)} + d_3 \\ &\vdots \\ x_n^{(k+1)} &= c_{n1}x_1^{(k+1)} + c_{n2}x_2^{(k+1)} + \dots + c_{nn-1}x_{n-1}^{(k+1)} + c_{nn}x_n^{(k)} + d_n \end{aligned}$$
$$x_i^{(k+1)} = \frac{b_i}{a_{ii}} - \sum_{j=1}^{i-1} \frac{a_{ij}}{a_{ii}} x_j^{k+1} - \sum_{j=i+1}^n \frac{a_{ij}}{a_{ii}} x_j^k \quad i = 1, 2, \dots, n$$
$$|x_1^{(k)} - x_1^{(k-1)}| \leq \varepsilon, \quad |x_2^{(k)} - x_2^{(k-1)}| \leq \varepsilon, \quad |x_3^{(k)} - x_3^{(k-1)}| \leq \varepsilon$$
$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|, \quad i = 1, 2, \dots, n$$

При этом хотя бы для одного уравнения неравенство должно выполняться строго.

Листинг программы

Полный код программы: <https://github.com/ulitsaRaskolnikova/comp-math/tree/lab1>

```
import numpy as np
import sys

INVALID_INPUT = 'Invalid input. Please try again.'
INVALID_FILE_DATA = 'Invalid file data.'
LEFT_BOUND = 0
RIGHT_BOUND = 100

def get_file_by_user():
    while True:
        try:
            filename = input('Enter the filename or press enter to use the
keyboard: ')
            if filename == '': return None
            file = open(filename, 'r')
            return file
        except: print(INVALID_INPUT)

def get_matrix_size_by_user():
    while True:
        try:
            n = int(input('Enter the matrix size: '))
            return n
        except: print(INVALID_INPUT)

def is_random_generated():
    while True:
        try:
            is_random = input('Do you want to generate a matrix with random
coefficients? [y/n]: ')
            if (is_random == 'y' or is_random == 'n'):
                return True if is_random == 'y' else False
        except: print(INVALID_INPUT)

def get_matrix_by_user(n):
    print('Enter the matrix: ')
    i = 0
    arr = []
    while i < n:
        try:
            line = list(map(float, input().split()))
            if len(line) == n:
                i += 1
                arr.append(line)
            else:
                print("Invalid length of line. Please try again.")
        except: print(INVALID_INPUT)
    return np.array(arr)
```

```

def get_values_by_user(n):
    while True:
        try:
            print('Enter the values: ')
            values = np.array(list(map(float, input().split())))
            if len(values) != n:
                print("Invalid length of values. Please try again.")
                continue
            return values
        except: print(INVALID_INPUT)

def get_accuracy_by_user():
    while True:
        try:
            accuracy = float(input('Enter the accuracy: '))
            return accuracy
        except: print(INVALID_INPUT)

def read_user_input():
    n = get_matrix_size_by_user()
    is_random = is_random_generated()
    matrix = None
    values = None
    if is_random:
        matrix = np.random.rand(n, n) * 100
        values = np.random.rand(n) * 100
        for i in range(n):
            matrix[i][i] = sum(matrix[i]) + 1
    else:
        matrix = get_matrix_by_user(n)
        values = get_values_by_user(n)
    accuracy = get_accuracy_by_user()
    return matrix, values, accuracy

def read_file_input(file):
    try:
        n = int(file.readline())
        matrix = np.array([list(map(float, file.readline().split())) for _ in
range(n)])
        values = np.array(list(map(float, file.readline().split())))
        accuracy = float(file.readline())
        return matrix, values, accuracy
    except:
        print(INVALID_FILE_DATA)
        return None, None, None

def read_data(file):
    if file is None: return read_user_input()
    return read_file_input(file)

```

```

def check_diagonal_dominance(matrix):
    n = len(matrix)
    for i in range(n):
        row_sum = sum(abs(matrix[i, j]) for j in range(n)) - abs(matrix[i, i])
        if abs(matrix[i, i]) < row_sum:
            return False
    return True

def swap_rows(matrix, b, i, j):
    matrix[[i, j]] = matrix[[j, i]]
    b[i], b[j] = b[j], b[i]

def make_diagonal_dominant(matrix, values):
    n = len(matrix)
    for i in range(n):
        if abs(matrix[i, i]) < sum(abs(matrix[i, j]) for j in range(n)) -
abs(matrix[i, i]):
            for j in range(i + 1, n):
                if abs(matrix[j, i]) > abs(matrix[i, i]):
                    swap_rows(matrix, values, i, j)
                    break
    if not check_diagonal_dominance(matrix):
        print('Matrix cannot be made diagonally dominant.')
        sys.exit()

def gauss_seidel(matrix, b, accuracy):
    n = len(matrix)
    x = np.zeros_like(b)
    iterations = 0
    while True:
        x_new = np.copy(x)
        for i in range(n):
            sigma = sum(matrix[i, j] * x_new[j] for j in range(n) if j != i)
            x_new[i] = (b[i] - sigma) / matrix[i, i]
        errors = abs(x_new - x)
        iterations += 1
        if max(errors) < accuracy:
            return x_new, iterations, errors
        x = x_new

def matrix_norm(matrix):
    return np.linalg.norm(matrix, ord=2)

file = get_file_by_user()
matrix, values, accuracy = read_data(file)
if matrix is None:
    sys.exit()

print(f"Initial Matrix:\n{matrix}")
print(f"Values: {values}")
print(f"Accuracy: {accuracy}")

```

```
make_diagonal_dominant(matrix, values)
print(f"Diagonal Dominant Matrix:\n{matrix}")

solution, iterations, errors = gauss_seidel(matrix, values, accuracy)
print(f"Solution: {solution}")
print(f"Number of iterations: {iterations}")
print(f"Errors: {errors}")
norm = matrix_norm(matrix)
print(f"Matrix norm: {norm}")
```

Примеры работы программы

Работа программы при вводе данных через файл матрицы с диагональным преобладанием:

Содержимое файла data.txt:

```
3
2 2 10
10 1 1
2 10 1
14 12 13
0.01
```

Вывод программы:

Enter the filename or press enter to use the keyboard: data.txt

Initial Matrix:

```
[[ 2.  2. 10.]
 [10.  1.  1.]
 [ 2. 10.  1.]]
```

Values: [14. 12. 13.]

Accuracy: 0.01

Diagonal Dominant Matrix:

```
[[10.  1.  1.]
 [ 2. 10.  1.]
 [ 2.  2. 10.]]
```

Solution: [0.9995552 1.00018016 1.00005293]

Number of iterations: 3

Errors: [0.0003552 0.00517984 0.00096493]

Matrix norm: 13.030989593863206

Работа программы при вводе из консоли матрицы без диагонального преобладания:

Работа программы:

Enter the filename or press enter to use the keyboard:

Enter the matrix size: 3

Do you want to generate a matrix with random coefficients? [y/n]: n

Enter the matrix:

1 2 3

1 2 3

1 2 3

Enter the values:

1 2 3

Enter the accuracy: 0.01

Initial Matrix:

[[1. 2. 3.]

[1. 2. 3.]

[1. 2. 3.]]

Values: [1. 2. 3.]

Accuracy: 0.01

Matrix cannot be made diagonally dominant.

Вывод

При выполнении лабораторной работы я познакомился с методом Гаусса-Зейделя и написал программу на Python, познакомившись с возможностями библиотеки numpy. Я углубил своё понимание этой темы, поняв алгоритм.